

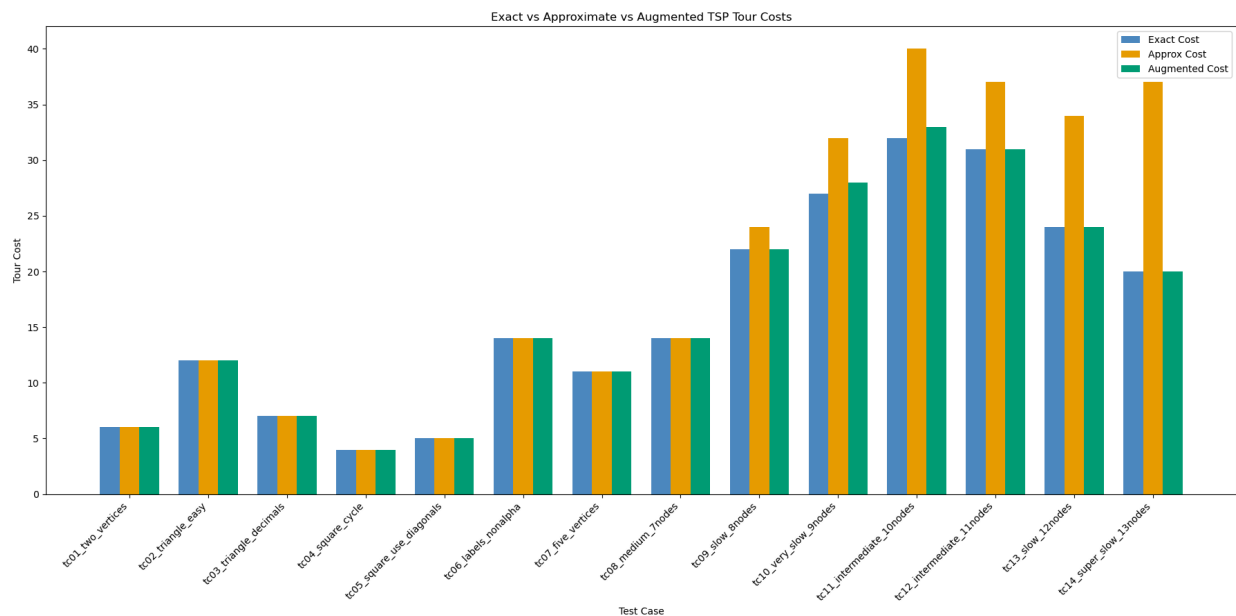
Abstract:

I liked the approximate approach of using a greedy algorithm, and also remembered Dr. Molloy saying in class that greedy is not always the best option, even that most of the time it is the worst option. I wanted to implement this approach and explore greedy some more and see if there was a way to optimize it.

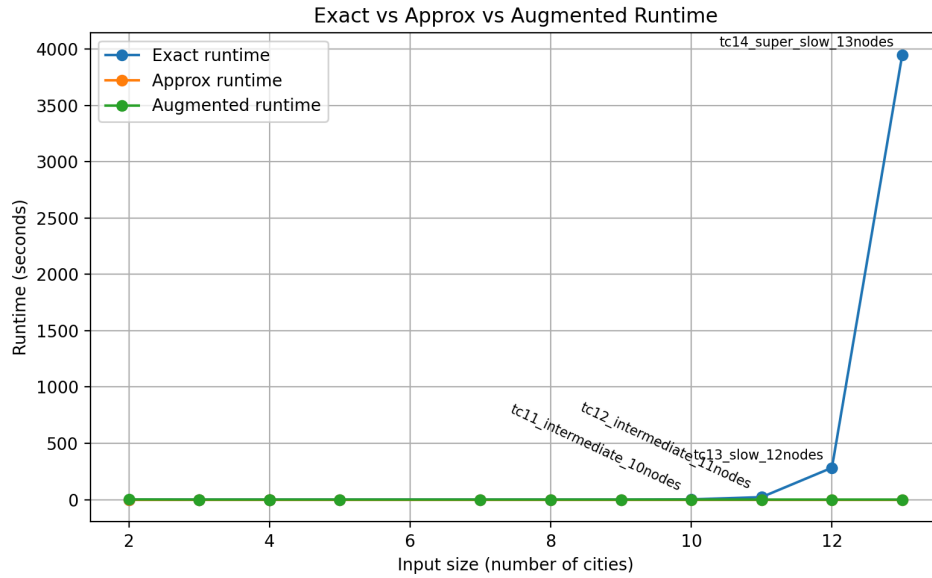
Strategy and Approach:

I use a multi restart with a nearest neighbor and a 2-opt local search. With each restart, the algorithm builds a greedy tour (deterministic nearest neighbor from a random start) and improves it with two_opt until there can't be anymore improvements or the time budget is met. It tracks the best tour and best cost over all restarts, so if a restart is less optimal, it keeps the best one found. The number of restarts depends on the time budget and the size of V . This approach allows me to get a good cost at the expense of a slightly slower algorithm.

In the test cases ran between the exact, approx, and augmented, the cost output was very similar between the augmented solution and the exact is very similar on input sizes less than 13. It is improved from the Approximate solution due to the two opt function.



The empirical runtime was greatly improved too, as you can see in the graph below. While the approximate and augmented run in roughly 0 seconds, you can really see the factorial performance from the exact solution, where as the other two are polynomial.



Analytical Runtime:

The TSP approximation work happens in `augment_tsp`. Let $\mathbf{R}$ be the number of restarts that fit into the time budget. Each restart builds a greedy tour in `greedy_tour` in $\mathbf{O}(V^2)$ time and then calls `two_opt`, whose worst-case runtime per call is $\mathbf{O}(V^3)$ (it considers $\mathbf{O}(V^2)$ edge pairs and each improving move may reverse a segment in $\mathbf{O}(V)$, and there can be up to $\mathbf{O}(V)$ such moves). Computing the cost of a tour with `tour_cost` is $\mathbf{O}(V)$. Thus one restart is $\mathbf{O}(V^3)$ in the worst case, and the overall approximation loop is $\mathbf{O}(\mathbf{R} \cdot V^3)$; in practice $\mathbf{R}$ is bounded by the time limit, so runtime is effectively capped by the time budget rather than by $\mathbf{E}$.